LABORATORIO 02

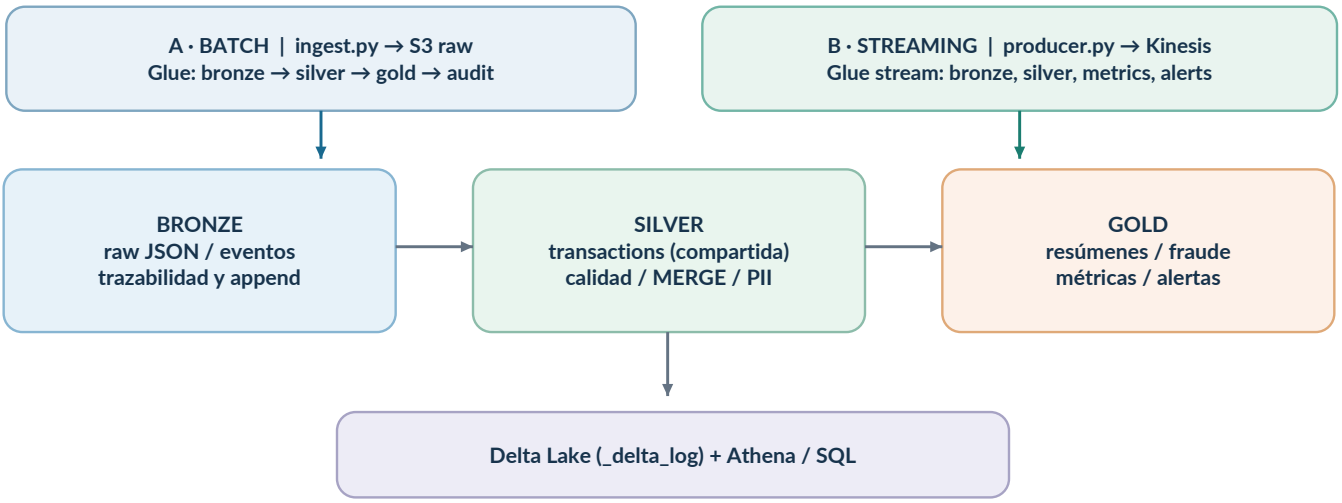
Lakehouse end-to-end en AWS

Caso: PayTrack Colombia | Fecha: 6 de noviembre de 2026 | Integrantes: Vladlen Shatunov y Juan Alberto Rodriguez

1) Objetivo y arquitectura

Se construyó un Lakehouse que integra **lotes diarios** y **eventos en tiempo real** en tablas Delta Lake sobre Amazon S3. AWS Glue ejecutó las transformaciones Bronze, Silver y Gold; Kinesis recibió el flujo de eventos y Athena consultó las salidas. Las validaciones y consultas se completaron y los recursos de AWS fueron eliminados después de las pruebas.

FLUJOS DE PROCESAMIENTO Y CONSUMO



Checkpoints S3 · CloudWatch · Control de versiones y time travel

2) Evidencia de ejecución y alcance

Control	Resultado verificado
Pruebas unitarias	15/15 PASSED (ingesta y productor)
AWS Glue Batch	Bronze, Silver, Gold y Audit exitosos
Streaming AWS	4.912 + 6.132 eventos; 11.044 acumulados
Silver final	40.562 transacciones; sin transaction_id duplicados
Consultas Athena	6/6 ejecutadas; métricas de escaneo registradas
Cierre operacional	Stack y stream Kinesis eliminados

Convención de evidencia. Se diferencia entre cifras observadas (capturas de Athena, CloudWatch y Delta) y escenarios estimados (latencia teórica y costos). Los archivos de resultados y logs se conservan como anexos de la entrega.

PARTE A · BATCH

Bronze, Silver y MERGE

Respuestas 1 a 3 · Idempotencia, calidad y actualización de estados

1. Bronze: esquema y reejecución

Bronze lee los campos como **texto** para preservar el dato original aunque el proveedor cambie tipos o envíe montos inválidos; Silver hace el casting y la validación. Con **replaceWhere por ingest_date**, al reprocesar un lote se reemplaza solo esa partición. Con **append**, el mismo JSON se añadiría otra vez, duplicando el lote en Bronze y afectando conteos y consumo. En nuestras ejecuciones repetidas, los controles mantuvieron el número de filas por fecha y la trazabilidad del archivo de origen.

2. Silver: cuarentena y orden de deduplicación

Lote	invalid_amount	missing_customer	invalid_status	Total
2026-09-29	1.176	594	0	1.770
2026-09-30	78	40	0	118

Resultado de Silver: lote 1, 28.230 transacciones válidas; lote 2, 1.883 nuevas y 275 cambios de estado. Las 1.770 y 118 filas rechazadas coinciden con el **GROUP BY ingest_date, dq_reason** de la cuarentena Delta local. Ninguna fila fue rechazada por invalid_status en esta muestra.

Reglas de calidad. missing_customer: customer_id nulo; invalid_amount: monto nulo, ≤ 0 o ≥ 100 millones; invalid_status: estado fuera del catálogo. Se registra el **primer motivo**. **La deduplicación se hace antes** de aplicar reglas: se conserva la versión de ingesta más reciente por transaction_id y se evita contar duplicados como rechazos separados o evaluar una versión antigua de la transacción.

3. MERGE: actualizar solo al cambiar el estado

El MERGE de Silver cruza por **transaction_id**: inserta las claves nuevas y modifica status, is_fraud y silver_updated_at únicamente cuando el estado cambia. Si se actualizara también cuando el status es igual, un reintento produciría otra escritura y cambiaría metadatos sin un cambio de negocio. La auditoría compara estados entre versiones: podría mostrar **0 cambios de status** aunque Delta registrara filas reescritas; además, su comprobación de equivalencia con numTargetRowsUpdated podría fallar.

Versión	Insertadas	Actualizadas	Copiadas
v2 · Batch 2	1.883	275	27.955
v3 · Repetición	0	0	30.113

Lectura. La versión 3 confirma idempotencia *lógica* (sin nuevas inserciones ni actualizaciones), pero el MERGE todavía copió 30.113 filas: la ejecución no fue físicamente gratuita.

PARTE A · BATCH

Auditoría, privacidad y consultas

Respuestas 4 a 6 · Time travel, SCD y rendimiento analítico

4. Auditoría con time travel y VACUUM

La auditoría del lote 2 comparó las versiones anterior y posterior de Silver, identificó **275 transacciones** cuyo estado pasó de approved a chargeback y un total revertido de **$19.660.867,66 COP**. Esos cambios se verificaron también en el TODO 4 de Athena (siete categorías de comercio). En la repetición del lote no aparecieron cambios nuevos. Delta registra versiones en _delta_log, pero **VACUUM elimina archivos físicos obsoletos** tras la retención configurada: una versión anterior puede dejar de poder leerse aun si su registro histórico existe. Para auditoría duradera habría que persistir cambios o ampliar retención.

5. PII: actualización Tipo 1 frente a SCD Tipo 2

En silver.customers se elimina name, se enmascara email y se **sobrescribe ciudad y segmento** con la información reciente (SCD Tipo 1). Así se pierde la clasificación histórica: por ejemplo, no puede reconstruirse exactamente cuánto consumía un cliente cuando pertenecía a otro segmento o vivía en otra ciudad. Como en el Laboratorio 1, se resolvería con **SCD Tipo 2** (surrogate key, valid_from, valid_to e is_current) y un join temporal de los pagos con la versión vigente en la fecha del evento.

6. Consumo Gold vs. Silver en Athena

Indicador	Gold · TODO 1	Silver · TODO 2
Datos escaneados	3,31 KB	99,68 KB
Tiempo de ejecución	1,398 s	1,663 s
Comercios recuperados	5	5
Monto aprobado	Coincidente	Coincidente

Gold leyó **30,1 veces menos datos** (reducción del 96,7 %) al usar agregaciones diarias precomputadas; Silver requirió recorrer transacciones individuales. Esto justifica Gold en consultas frecuentes, con menos latencia potencial y consumo de lectura. A cambio se pierde granularidad para nuevas preguntas, se depende del refresco de agregados y pueden aparecer diferencias pequeñas por redondeo: en M002, la tasa fue 98,89 % en Gold frente a 98,94 % en Silver. Una ejecución aislada no permite concluir una ventaja estable de tiempo.

Resultado adicional. En TODO 3, el máximo semanal de fraude fue: mass (13 eventos, semana del 21/09); high_value (8, semana del 14/09); premium (7, semana del 21/09).

PARTE B · STREAMING

Entrega e información tardía

Respuestas 7 y 8 · Reintentos, checkpoints y watermark

7. Garantías de entrega e idempotencia

El productor implementa **at-least-once**: reintenta solicitudes fallidas de envío y, si no recibe confirmación, puede retransmitir un evento que el broker ya aceptó. Kafka y Kinesis no ofrecen por sí solos un resultado exactamente una vez de extremo a extremo. En el lakehouse, el **checkpoint por consulta** recuerda offsets procesados, **Delta** aporta escrituras transaccionales, la deduplicación se apoya en event_id y **MERGE por transaction_id** evita duplicar entidades en Silver. Esta combinación produce un estado final *efectivamente idempotente*, sin prometer exactly-once universal ante todas las fallas.

Evidencia: en la repetición local de availableNow sin eventos nuevos se leyeron 0 filas; en Batch 2 repetido hubo 0 inserciones y 0 actualizaciones; Silver terminó con claves únicas. Bronze conserva eventos como llegaron, por lo que la garantía se analiza por capa.

8. Watermark: comparación del experimento local

Modo / prueba	Eventos procesados	Descartes Metrics / Alerts
availableNow · fast	12.241	8 / 8
En vivo · 20/s	2.446	40 / 40
En vivo · 50/s	6.180	124 / 124

Las consultas metrics y alerts emplean un watermark de **2 minutos** sobre event_time. En availableNow los datos acumulados se consumen en tandas; en vivo, el máximo event_time avanza con el procesamiento y los registros atrasados pueden superar el umbral. Por ello los contadores son distintos aun con proporciones similares de tardíos. Las cifras corresponden a **descartes reportados por cada consulta**, no necesariamente a 8, 40 o 124 eventos únicos globales.

Silver no usa watermark porque necesita conservar pagos tardíos y contracargos para reconstruir el estado final. Si lo aplicara, un pago válido muy tardío podría descartarse y no incorporarse en Silver; un watermark es adecuado para acotar el estado de las agregaciones, no para decidir qué transacciones existen.

Contraste AWS: la recuperación a 20/s descartó 0 filas; la ejecución en vivo configurada a 50/s reportó 43 descartes por watermark en metrics y 43 en alerts. No son mediciones directamente equivalentes a las pruebas locales.

PARTE B · STREAMING

Latencia y precisión de alertas

Respuestas 9 y 10 · Ventanas, tiempo de evento y carga

9. Tiempo hasta que aparece una alerta

Una alerta se calcula para cada **ventana deslizante de 30 s, avance de 10 s**, con al menos seis pagos de un cliente. La salida está en modo **append**: Spark solo publica la ventana cuando el watermark la da por cerrada. Con **2 minutos** de tolerancia, la publicación puede retrasarse hasta que el máximo event_time observado supere el fin de ventana en 120 s, más la espera del siguiente micro-lote (10 s) y el tiempo de escritura a Delta. No es un temporizador absoluto: si el stream se detiene, el watermark puede no avanzar.

Medición y límite: no se preservó un par de marcas de tiempo de "último pago de ráfaga → primera alerta disponible"; por eso **no se reporta una latencia observada**. Para disminuirla, se podría acortar el watermark, emitir actualizaciones provisionales o usar un detector de reglas de estado más inmediato. Se sacrificarían tolerancia a eventos tardíos, simplicidad del modo append o estabilidad de resultados.

10. Precisión de detección: 20 frente a 50 eventos/s

Indicador	Prueba 20/s	Prueba 50/s
Eventos enviados	4.912	6.132
Alertas / aciertos	17 / 16	60 / 40
Falsos positivos	1	20
Precisión	94,12 %	66,67 %
Watermark (metrics / alerts)	0 / 0	43 / 43

Precisión observada en Athena (TODO 5)



La caída de **27,45 puntos porcentuales** coincide con más alertas falsas en la segunda ejecución. Sin embargo, el primer ensayo recuperó mensajes ya acumulados en Kinesis y el segundo procesó en vivo una tasa configurada de 50/s (tasa efectiva ≈38,6/s). Por ello **no puede atribuirse la diferencia solo al caudal**. Para mejorar precisión: exigir variedad de comercios o montos anómalos, combinar la ráfaga con riesgo del cliente, calibrar el umbral y contrastar con recall y falsos negativos.

PARTE B · STREAMING

Estados finales y Kafka

Respuestas 11 y 12 · Contracargos, orden y particionamiento

11. Por qué el contracargo debe ser terminal

El productor emite reenvíos y eventos tardíos de pago. Caso concreto: una transacción recibe **payment → chargeback**, pero después se reenvía el payment con estado approved. Sin la protección que impide actualizar una fila ya en **chargeback**, el MERGE podría restaurar approved y "deshacer" un contracargo de negocio. **latest_per_transaction** conserva una fila por transaction_id *dentro de cada micro-lote*; no compara necesariamente los micro-lotes futuros con todo el historial. El estado terminal requiere una regla persistente en el MERGE, independiente del orden de llegada.

12. Experimento de partition key en Kafka

Clave usada	Part. 0	Part. 1	Part. 2	Mensajes
Constante	1.238	0	0	1.238
transaction_id	392	411	416	1.219
customer_id (topic original)	7.870	6.354	6.643	20.867

La **clave constante** concentra el 100 % en P0, creando un cuello de botella; **transaction_id** reparte casi uniformemente (32,2 %, 33,7 % y 34,1 %). customer_id también usa las tres particiones, aunque con más sesgo. Los dos topics experimentales contienen números de eventos distintos, por lo que se comparan distribuciones porcentuales, no cargas absolutas.

¿Cambiarían las alertas? No necesariamente: Spark vuelve a agrupar por customer_id y puede reordenar entre particiones. Pero el reparto sí afecta throughput, retardo, paralelismo y, de forma indirecta, ventanas y eventos tardíos. La partition key mantiene **orden relativo por clave dentro de la partición**; usar transaction_id reúne pago y contracargo de la misma transacción, mientras customer_id reúne eventos de un cliente, lo que puede ayudar a su patrón temporal.

Prueba de productor: pytest verificó que el contracargo y su pago usan la misma partición, la distribución de claves y los reintentos selectivos de Kinesis. Resultado: 15/15 pruebas aprobadas.

Delta Lake compartido

Respuesta 13 · Historial MERGE, concurrencia y fuente de verdad

13. Batch y Streaming escribiendo sobre la misma tabla

El historial de Delta de silver.transactions alcanzó la **versión 8** (versiones 0–8) y registra **ocho MERGE**. Estos son los contadores reales obtenidos desde _delta_log:

Versión	Insertadas	Actualizadas	numTargetRowsCopied
1 · Batch 1	28.230	0	0
2 · Batch 2	1.883	275	27.955
3 · Repetición	0	0	30.113
4 · Streaming	4.645	100	30.013
5 · Streaming	2.605	61	30.052
6 · Streaming	2.120	41	32.677
7 · Streaming	735	17	34.821
8 · Streaming	344	7	35.566
Total	40.562	501	221.197

El crecimiento acumulado de filas copiadas (221.197) evidencia **amplificación de escritura**: Delta puede reescribir archivos Parquet enteros aunque el micro-lote modifique pocas filas. A 500.000 pagos/día, convienen particiones adecuadas, compactación, clustering/optimización, control del tamaño de micro-lotes y evitar actualizar registros inalterados. La versión 3 copió 30.113 filas aunque actualizó 0.

Concurrencia. Si Batch Silver y Streaming ejecutan MERGE simultáneos en los mismos archivos, las transacciones Delta podrían entrar en conflicto de control optimista: una escritura se debe reintentar contra la versión reciente; una estrategia operacional sería separar ventanas de actualización o serializar escritores por tabla. No se debe confiar en el orden físico de llegada para resolver estados finales.

TODO 6 (capa rápida vs. verdad). Para la fecha observada, Gold sumó **2.755 pagos** en ventanas publicadas frente a **4.645** transacciones de stream en Silver: diferencia de **1.890** (cobertura Gold 59,3 %). Silver es la fuente de verdad del estado transaccional; Gold aporta baja latencia y agregados potencialmente incompletos por ventanas no cerradas.

OPERACIÓN Y CONCLUSIONES

Costos y decisiones operativas

Respuesta 14 · Escenario: 500.000 transacciones/día · Región de referencia: us-east-2

14. Estimación mensual de la solución completa

Para **15 millones de eventos/mes** (500.000 por día durante 30 días), se modelan cuatro jobs Batch diarios de 2 DPU y 10 min cada uno; Streaming Glue de 2 DPU en operación 24/7; Kinesis provisionado de 2 shards, un PUT de ≤25 KB por evento; 45 GB medios almacenados en S3 entre capas/versiones; y 50 consultas Athena diarias que escanean 1 GB cada una. Estos volúmenes y duraciones son **supuestos de presupuesto**, no mediciones de escala.

Servicio	Supuesto y cálculo mensual	Estimado USD
S3 Standard	45 GB × USD 0,023 + solicitudes aprox.	1,22
Glue Batch	4 jobs × 2 DPU × 10 min × 30 d × 0,44/h	17,60
Glue Streaming 24/7	2 DPU × 720 h × USD 0,44/h	633,60
Kinesis provisionado	2 × 720 h × 0,015 + 15 M PUT × 0,014/M	21,81
Athena	50 × 30 × 1 GB ≈ 1,5 TB × USD 5,65/TB	8,48
TOTAL ORIENTATIVO	Sin impuestos, transferencia ni sobrecostos	682,71

Qué cambiaría sin streaming 24/7. Si se necesitara procesar solo 8 h/día, Glue Streaming costaría alrededor de **USD 211,20/mes**, ahorro de USD 422,40 respecto a 24/7. Kinesis seguiría cobrando mientras sus shards existan; se necesitaría también un plan de apagado/creación de streams, consumo bajo demanda o ejecución en micro-batch. El promedio del escenario es solo 5,8 eventos/s; 2 shards pueden estar sobredimensionados fuera de picos.

Por qué no expirar lakehouse/ con una regla S3. Delta requiere integridad entre **_delta_log** y los archivos de datos referenciados. Una regla general de expiración borraría versiones o Parquet aún necesarios y podría romper consultas, actualizaciones y time travel. La retención debe administrarse mediante **VACUUM controlado**, políticas de snapshots/auditoría y archivado selectivo en zonas apropiadas (por ejemplo, raw o respaldos), no con borrado indiscriminado.

Síntesis y fuentes

Conclusión. La arquitectura integró 2 lotes, 11.044 eventos de Kinesis y seis consultas Athena. Gold redujo lectura 96,7 %, Silver preservó idempotencia lógica y las alertas mostraron precisión 94,12 % y 66,67 % en dos condiciones experimentales diferentes. La principal mejora futura es reducir reescritura Delta y latencia de ventanas sin perder consistencia.

Referencias. Repositorio SI7006 (Lab 02); capturas y CSV de Athena; CloudWatch; historial Delta local/AWS. Tarifas orientativas: aws.amazon.com/glue/pricing/ · aws.amazon.com/kinesis/data-streams/pricing/ · aws.amazon.com/s3/pricing/ · aws.amazon.com/es/athena/pricing/.